

(A) 3 (B) 15 (C) 9 (D) -3

- (e) [2 points] What is the maximum number of swaps that can be performed in the Selection Sort algorithm?

(A) n (B) $n - 1$ (C) $n - 2$ (D) 1

- (f) [2 points] Let A be a square matrix of size $n \times n$. Consider the following algorithm. What is the expected output?

```
C = 500
for i = 1 to n do
    for j = 1 to n do {
        Temp = A[i][j] + C
        A[i][j] = A[j][i]
        A[j][i] = Temp - C
    }
for i = 1 to n do {
    for j = 1 to n do
        Output(A[i][j]);
    Output("\n");
}
```

(A) Transpose of matrix A

(B) The matrix A itself

(C) Adding 500 to the diagonal elements

(D) Adding 500 to the diagonal elements and subtracting 500 from anti-diagonal elements of A

- (g) [2 points] Suppose a circular queue of capacity $(n - 1)$ elements is implemented with an array of n elements. Assume that the insertion and deletion operation are carried out using REAR and FRONT as array index variables, respectively. Initially, $\text{REAR} = \text{FRONT} = 0$. The conditions to detect queue **Full** and queue **Empty** are

(A) **Full:** $(\text{FRONT} + 1) \bmod n == \text{REAR}$, **Empty:** $\text{REAR} == \text{FRONT}$

(B) **Full:** $(\text{REAR} + 1) \bmod n == \text{FRONT}$, **Empty:** $(\text{FRONT} + 1) \bmod n == \text{REAR}$

(C) **Full:** $\text{REAR} == \text{FRONT}$, **Empty:** $(\text{REAR} + 1) \bmod n == \text{FRONT}$

(D) **Full:** $(\text{REAR} + 1) \bmod n == \text{FRONT}$, **Empty:** $\text{REAR} == \text{FRONT}$

- (h) [2 points] A three dimensional array in C language is declared as $\text{int } A[x][y][z]$. Here, the address of an item at the location $A[p][q][r]$ can be computed as follows (where w is the word length of an integer):

(A) $\&A[0][0][0] + w(y * z * p + x * q + r)$

(B) $\&A[0][0][0] + w(x * y * p + y * q + r)$

(C) $\&A[0][0][0] + w(y * z * p + z * q + r)$

(D) $\&A[0][0][0] + w(x * y * z * p + y * z * q + r * z)$

4. [5 points] Given a collection of algorithms that runs on $O(n^{10})$, $O(n^{2n})$, $O(10000)$, $O(\log n)$, $O(n!)$, $O(n \log n)$, $O(n)$, $O(n \log \log n)$ order the algorithms **from fastest to slowest** when n very large.
5. [6 points] Given a circular queue of size 4 (i.e., $\text{MAX_QUEUE_SIZE} = 4$), what is the **front** and **rear** values become when we do the following commands?
(Initial circular queue setting: $\text{front} = \text{rear} = 0$)
add A, add B, add C, add D, delete, add E, add F, add G, delete, delete, add H, add I, add J
6. [10 points] Use **Big-Theta** notation,
- (a) [3 points] Suppose the recursive function $T(n)$ calls $T(n-1)$ five times within its body. Also assume the recursion would end when $n=0$. Estimate the time complexity of $T(n)$.

For the following question (b)~(c),

```
int i, j, k = 0;
for (i = n / 2; i <= n; i++)
    for (j = 2; j <= n; j = j * 2)
        k = k + n / 2;
```

Code 1

```
int a = 0, i = N;
while (i > 0) {
    a += i;
    i /= 2;
}
```

Code 2

- (b) [3 points] Estimate the time complexity of **Code 1**.
- (c) [4 points] Estimate the time complexity of **Code 2**.
7. [10 points] Consider the following recursive functions.

```
int func1(int a, int b){
    if (a % b == 0)
        return b;
    else
        return func1(b, a % b);
}
```

```
int func2(int a, int b){
    if (a==0)
        return b+1;
    else
        if (b==0)
            return func2(a-1, 1);
        else
            return func2(a-1, func2(a, b-1));
}
```

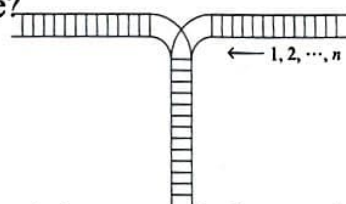
- (a) [5 points] What is the output given by $\text{func1}(90, 52)$?
- (b) [5 points] What is the output given by $\text{func2}(2, 3)$?
8. [10 points] What are the values of **a** to **j** in the following programs:

Statement	s/e	Frequency
<code>void func1(int a[][n], int b[][n], int d[][n])</code>		
<code>{</code>		
<code>int i, j, k;</code>		

for (i = 0; i < n; i++)	1	a
for (j = 0; j < n; j++) {	1	b
c [i][j] = 10;	1	c
for (k = 0; k < n; k++)	1	d
d [i][j] += a [i][k] * b [k][j] - 20;	1	e
}		
}		
Total steps = $\Theta(f)$		

Statement	s/e	Frequency
float func2(float list[], int n)		
{		
If (n)	1	g
return func2(list, n-1) * list[n-1];	1	h
return list[0];	1	i
}		
Total steps = $\Theta(j)$		

9. [11 points] Consider the railroad switching network. Railroad cars numbered 1, 2, ..., n are at the right. Each car is brought into the stack and removed at any time.



- (a) [2 points] For $n = 9$, is the new order 3 4 5 2 1 8 9 6 7 possible?
- (b) [2 points] How many possible permutations for $n = 4$.
- (c) [3 points] How many possible permutations for $n = 5$?
- (d) [4 points] How many possible permutations for $n = 6$?
10. [12 points] In C/C++ language, the standard way of writing c expressions is known as **infix** notation because in it we place a binary operator in-between its two operands. Instead compilers typically use a parenthesis-free notation referred to as **postfix**. Another expression form that is easy to evaluate and is parenthesis-free is known as **prefix**. In Prefix notation, the operators precede their operands.

- (a) [4 points] Write the **postfix** form of the infix expressions:

$A + B / C * (D - A) / F / H - E * G$

- (b) [5 points] Write the **prefix** form of the postfix expressions:

$C \mid B \mid A > D \mid E < \mid \mid \&\& \mid C \mid E < \mid \mid$

- (c) [3 points] What is the value of the following postfix expression?

$6 \ 3 \ 4 \ * \ + \ 2 \ / \ 2 \ 8 \ 2 \ + \ * \ 8 \ * \ -$